

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»
Филиал
«Минский радиотехнический колледж»

Учебный предмет
«Разработка кроссплатформенных приложений»

Отчёт
по выполнению лабораторной работы
«Разработка, отладка и испытание алгоритмов и программ с применением
мьютексов»

Выполнил: учащийся гр. 3к9391
Шутько А.П.
Проверил: Грунтович П.Н.

Минск 2025

Лабораторная работа № 19

Тема работы: «Разработка, отладка и испытание алгоритмов и программ с применением мьютексов»

1. Цель работы

Сформировать умение ограничение доступа к общему ресурсу на базе мьютекса.

2. Задание

Номер варианта соответствует вашему номеру по списку, если номер по списку больше последнего варианта, то необходимо начать подсчет варианта с начала (например, учащийся с номером по списку 25 должен выполнить 2 вариант в первом задании и 3 вариант во втором задании).

3. Оснащение работы

Задание по варианту, ЭВМ, среда разработки.

4. Выполнения работы

Задание 1: Производители помещают в буфер сообщения. Потребители считывают эти сообщения и выводят на экран. Есть n производителей и m потребителей. Разделяемый буфер управляется двумя операциями: `deposit` (поместить) и `fetch` (извлечь). Выполняя операцию `deposit`, производители помещают сообщения в буфер, потребители получают сообщения при помощи операции `fetch`. Чтобы сообщения не перезаписывались и каждое из них могло быть получено только один раз, выполнение операций `deposit` и `fetch` должно чередоваться, причем первой должна быть операция `deposit`.

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.concurrent.Semaphore;

class SharedBuffer {
    private Queue<String> buffer = new LinkedList<>();
    private Semaphore semDeposit = new Semaphore(1);
    private Semaphore semFetch = new Semaphore(0);

    public void deposit(String msg, int id) {
        try {
            semDeposit.acquire();
            buffer.add(msg);
        }
    }
}
```

```

        System.out.println("Producer-" + id + " put: " + msg);
        semFetch.release();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

public void fetch(int id) {
    try {
        semFetch.acquire();
        String msg = buffer.poll();
        System.out.println("Consumer-" + id + " got: " + msg);
        semDeposit.release();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

class Producer extends Thread {
    private SharedBuffer buffer;
    private int id;
    private String[] msgs = {"A", "B", "C", "D", "E"};

    public Producer(SharedBuffer buffer, int id) {
        this.buffer = buffer;
        this.id = id;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            buffer.deposit(msgs[i] + "_from_P" + id, id);
            try { sleep(100); } catch (InterruptedException e) {}
        }
    }
}

class Consumer extends Thread {
    private SharedBuffer buffer;
    private int id;

    public Consumer(SharedBuffer buffer, int id) {
        this.buffer = buffer;
        this.id = id;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            buffer.fetch(id);
            try { sleep(150); } catch (InterruptedException e) {}
        }
    }
}

public class Main {
    public static void main(String[] args) {
        int n = 3;
        int m = 2;
    }
}

```

```

        SharedBuffer buffer = new SharedBuffer();

        Producer[] producers = new Producer[n];
        Consumer[] consumers = new Consumer[m];

        for (int i = 0; i < n; i++) {
            producers[i] = new Producer(buffer, i + 1);
            producers[i].start();
        }

        for (int i = 0; i < m; i++) {
            consumers[i] = new Consumer(buffer, i + 1);
            consumers[i].start();
        }

        try {
            for (int i = 0; i < n; i++) {
                producers[i].join();
            }
            for (int i = 0; i < m; i++) {
                consumers[i].join();
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("--- All done ---");
    }
}

```

```

Producer-3 put: A_from_P3
Consumer-1 got: A_from_P3
Producer-2 put: A_from_P2
Consumer-2 got: A_from_P2
Producer-1 put: A_from_P1
Consumer-1 got: A_from_P1
Producer-3 put: B_from_P3
Consumer-2 got: B_from_P3
Producer-1 put: B_from_P1
Consumer-1 got: B_from_P1
Producer-2 put: B_from_P2
Consumer-2 got: B_from_P2
Producer-3 put: C_from_P3

```

Рисунок 1 – Результат работы программы

5. Контрольные вопросы и задания

1. Для чего применяются синхронизаторы?

Синхронизаторы применяются для координации работы нескольких потоков, позволяя управлять порядком их выполнения, доступом к общим ресурсам и обменом данными между ними. Они решают типовые задачи многопоточного программирования, такие как ограничение количества потоков, одновременно работающих с ресурсом, организация ожидания достижения определенного состояния всеми потоками, обеспечение взаимного исключения при доступе к критической секции, а также синхронизация производителей и потребителей. В Java к синхронизаторам относятся семафоры, мьютексы, защелки, барьеры, обменники и другие средства из пакета `java.util.concurrent`.

2. Что представляют собой защелки?

Защелки (`Latch`) представляют собой синхронизаторы, которые позволяют одному или нескольким потокам ожидать, пока в других потоках не будет завершена определенная совокупность операций. Классическая реализация в Java – `CountDownLatch`, который инициализируется заданным счетчиком. Потоки, вызывающие метод `await()`, блокируются до тех пор, пока счетчик не станет равным нулю. Другие потоки вызывают метод `countDown()`, уменьшая счетчик на единицу. Особенность защелки в том, что после достижения счетчиком нуля она не может быть сброшена или использована повторно – все последующие вызовы `await()` проходят без блокировки. Защелки применяются для запуска группы потоков одновременно после завершения подготовительных операций или для ожидания завершения всех подзадач в основном потоке.

3. В каких случаях лучше всего использовать `FutureTask`?

`FutureTask` лучше всего использовать в случаях, когда необходимо выполнить длительную или ресурсоемкую асинхронную операцию и получить ее результат в будущем, при этом имея возможность контролировать выполнение и отменять его при необходимости. Особенно удобен `FutureTask` для организации кэширования результатов дорогостоящих вычислений – задача выполняется только один раз при первом запросе, а все последующие вызовы `get()` мгновенно получают сохраненный результат. Также его применяют при параллельных вычислениях, когда один поток выполняет основную работу, а другой ожидает результат; для реализации таймаутов при ожидании результата; для выполнения задач в пуле потоков с последующим объединением результатов. `FutureTask` часто используется вместе с `ExecutorService` для асинхронной обработки данных.

4. Опишите мьютексы. Когда удобнее всего их применять?

Мьютекс (`mutex`, `mutual exclusion`) – это синхронизатор, обеспечивающий взаимное исключение, то есть гарантирующий, что только один поток в единицу

времени может выполнять критическую секцию кода или обращаться к защищенному ресурсу. В Java мьютекс реализован ключевым словом `synchronized` (на неявном мониторе каждого объекта) и явным классом `ReentrantLock`. Мьютекс поддерживает состояние занят/свободен и может переводить поток в ожидание, если блокировка уже захвачена другим потоком. Удобнее всего применять мьютексы для защиты разделяемых данных от состояния гонки (`race condition`), при работе с общими коллекциями и счетчиками, при реализации атомарных операций над сложными структурами данных, а также везде, где требуется гарантировать, что фрагмент кода выполняется не более чем одним потоком одновременно. В отличие от семафора со счетчиком 1, мьютекс часто поддерживает понятие владельца блокировки (только поток, захвативший мьютекс, может его освободить).

5. Какое назначение имеют барьеры?

Барьеры (`Barrier`) предназначены для синхронизации группы потоков, выполняющих некоторую работу параллельно, таким образом, что все потоки должны достичь определенной точки (барьера), прежде чем любой из них сможет продолжить выполнение. В Java основная реализация – `CyclicBarrier`, которая позволяет задать количество потоков-участников и опциональное действие, выполняемое после достижения барьера всеми потоками. Когда каждый поток вызывает метод `await()`, он блокируется до тех пор, пока все остальные потоки не вызовут `await()` и не достигнут барьера. Отличие барьера от защелки в том, что барьер можно использовать многократно (циклически) – после прохождения барьера счетчик сбрасывается, и потоки могут ждать следующей фазы. Барьеры применяются в многофазных алгоритмах, например, при численном моделировании на сетках, при параллельной сортировке слиянием, в итеративных алгоритмах, где каждая итерация требует синхронизации всех потоков перед началом следующей.